Day - 4

07 May 2026 09:34

1. Openshift security Model.
2. Access Control Deep Dive.
3. Scalability & Reliability
4. Health check.
5. Monitoring & Logging.

1. openshift security Model :-

i) SCC (Security Context constraints)
ii) Running containers as non-root
iii) Security image Practices.
iv) Trusted Registry.

1. **Why?**

Containers run in host OS Kernel

Not configured properly, it arises.

Reduce :-

1) SCC
2) RBAC
4) Image Security
5) Secret Mgt.
6) Non-root containers

(2) RBAC
(3) SA
(6) Non-root container
enf.

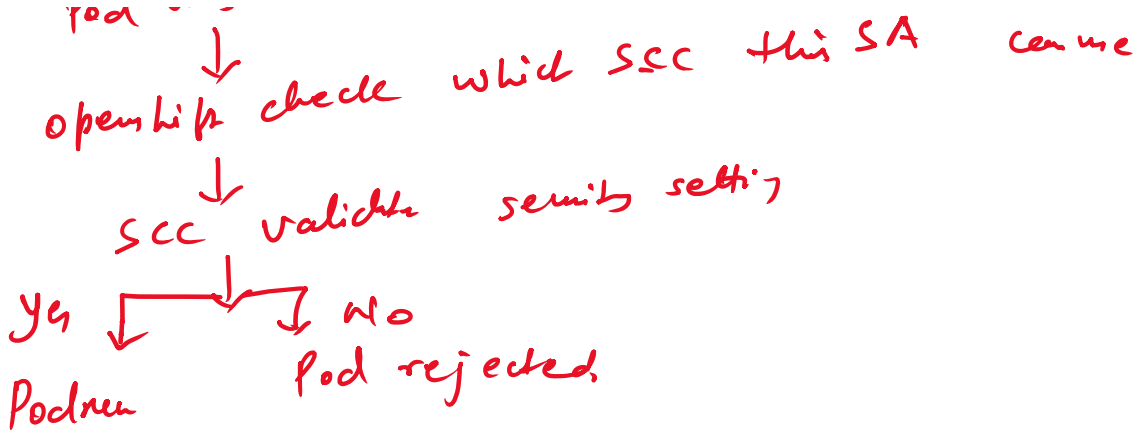
User applies Pod yaml
↓
openshift check user permission
↓
openshift check SA permission
↓
SCC admission check security rules
↓
Pod is allowed or rejected
↓
Container runs with restricted permission.

SCC :— Security Constraint Context.

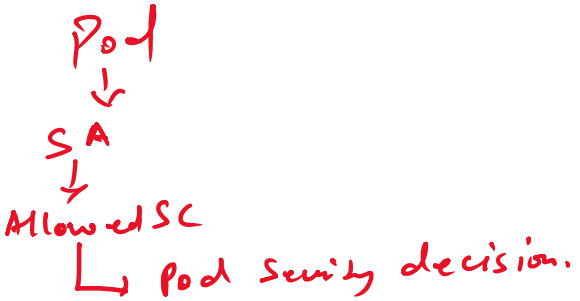
SCC = Security Rulebook of the Pod

Decide?
(1) Run as root or non-root
(2) Use privilege
(3) Host ports
(4) Host networking
(5) Mount hostpath volume
(6) Linux capabilities
(7) Specific filesystem group

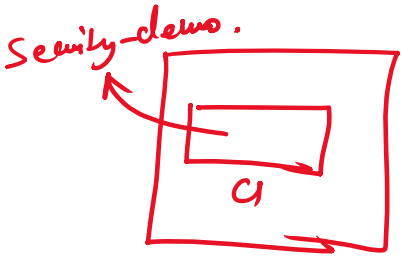
Developer create Pod
↓
Pod uses SA
↓
check which SCC this SA can use



① SA & SCC :-




```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: security-nonroot-pod
  labels:
    app: security-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      while true;
      do
        echo "Container is running";
        echo "Current UID:";
        id;
        sleep 30;
      done
  restartPolicy: Never
EOF
```



Pod

Security-nonroot-Pod

infinite loop

2 → interval of 30sec.

↳ Container/Pod crash. It will not create new Pod automatically.

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: security-root-pod
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      id;
      sleep 3600;
    securityContext:
      runAsUser: 0
  restartPolicy: Never
```

EOF

Practice for Image Creation —

1. Not require root
2. Not write to root - owned directories
3. Use Minimal base image
4. Run only req package.

```
FROM registry.access.redhat.com/ubi9/ubi-minimal
WORKDIR /app
COPY app.sh /app/app.sh
RUN chmod +x /app/app.sh
CMD ["/app/app.sh"]
```

→ Base Image

UBI Mini BI → Create/Use → Copy → Make → Run

(1) Random non-root UID.

```
FROM registry.access.redhat.com/ubi9/ubi-minimal
WORKDIR /app
COPY app.sh /app/app.sh
RUN chmod +x /app/app.sh && chmod -R g=u /app
USER 1001
CMD ["/app/app.sh"]
```

→ ensure it will run a non-root

Practice —

1. Small
2. Mini.
3. Scanned
4. Updated
5. Non-root
6. Tags
7. Free hardcoded Sensitive data
8. Build with least Privileges.

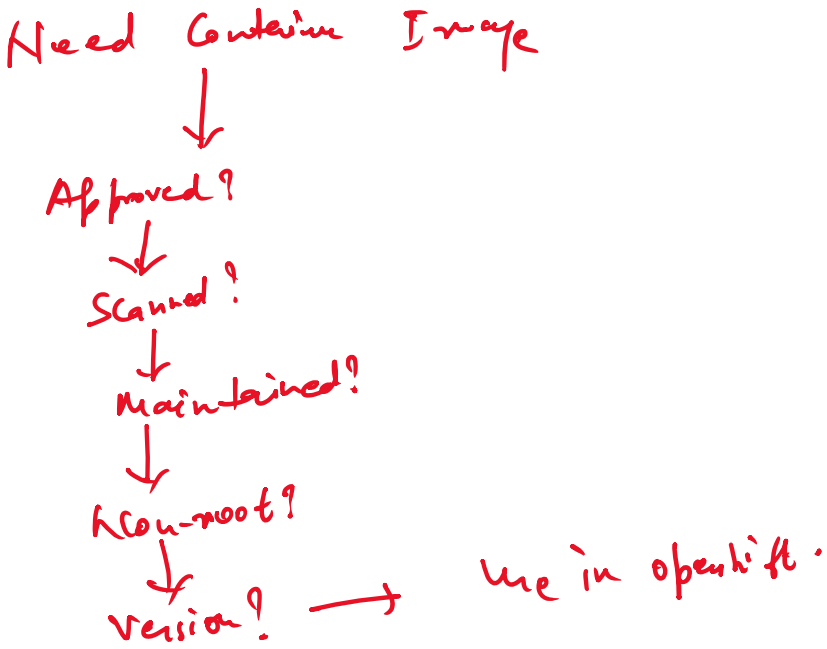
```
image: nginx:latest
```

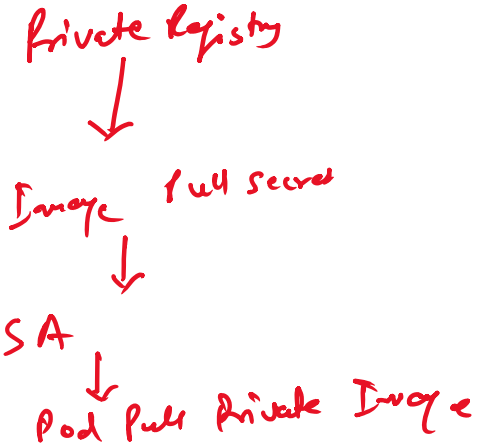
image: registry.redhat.io/ubi9/nginx-120:1-86

image: quay.io/myorg/myapp:v1.2.3

Trusted Registry :- Approved by org.

registry.redhat.io
registry.access.redhat.com
quay.io/company-approved-org
private enterprise registry
OpenShift internal registry





(2.) Scalability & Reliability

HPA - Horizontal Pod Autoscaling.

(2) Scaling

(i) HPA – Horizontal Pod Autoscaling.

(ii) Resource req. & limit

(iii) High Availability concept

(iv) Pod disruption Basic

User Traffic

↓

Route

↓

Service

↓

Multiple Pods ⟶ if one Pod fails, other continue serving.

HPA (Horizontal Pod Autoscaler) :-

No. of Pod ∝ Resource Usage.

CPU↑ ∝ increase Pod

CPU↓ ∝ decrease Pod.

Deployment + HPA

HPA flow :-

Metrics / serv

↓

HPA check CPU

↓

Target

Replica count

↳ Deployment create Pod.

Current Pod = 1

CPU target = 50%

Actual cpu = 90%

HPA = Pod 2 or 3

Request = 50mb

Limit = 100mb

```
cat <<'EOF' | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: resource-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: resource-demo
  template:
    metadata:
      labels:
        app: resource-demo
    spec:
      containers:
      - name: app
        image: registry.access.redhat.com/ubi9/ubi-minimal:latest
        command: ["/bin/sh", "-c"]
        args:
        - |
          while true;
          do
            echo "Resource demo running";
            sleep 30;
          done
        resources:
          requests:
            cpu: "50m"
            memory: "64Mi"
          limits:
            cpu: "100m"
            memory: "128Mi"
EOF
```

Deployment

↓

replicaSet

↓

Pod

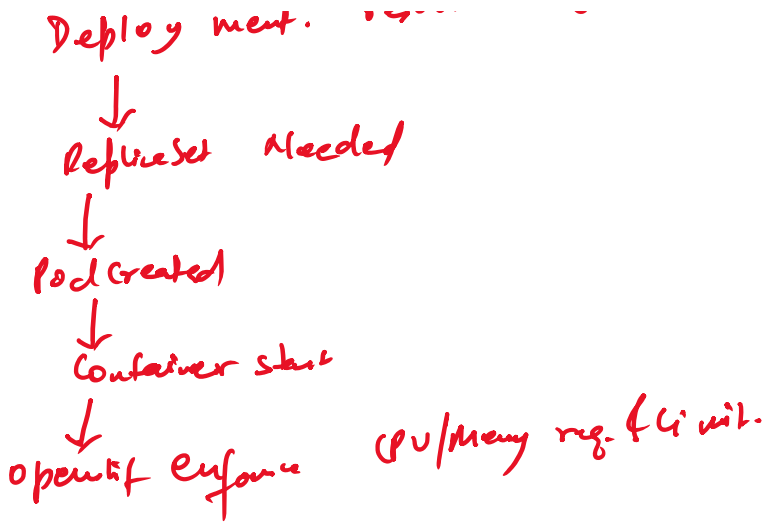
↓

Container.

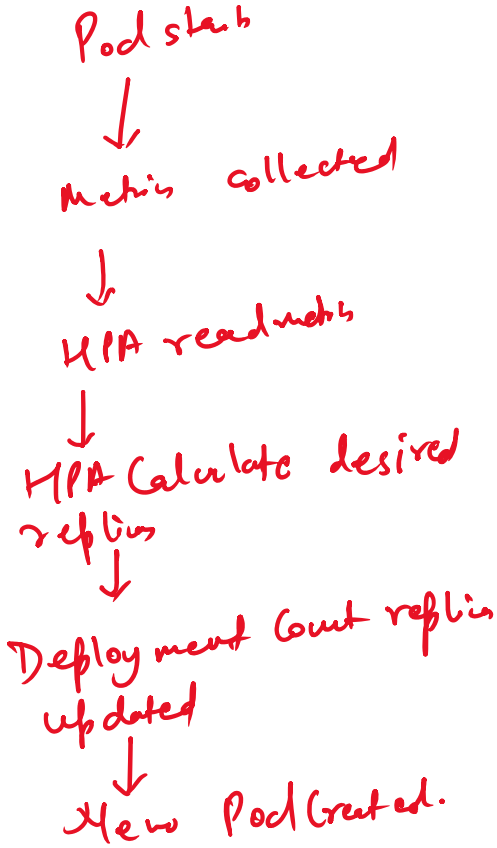
OC Apply

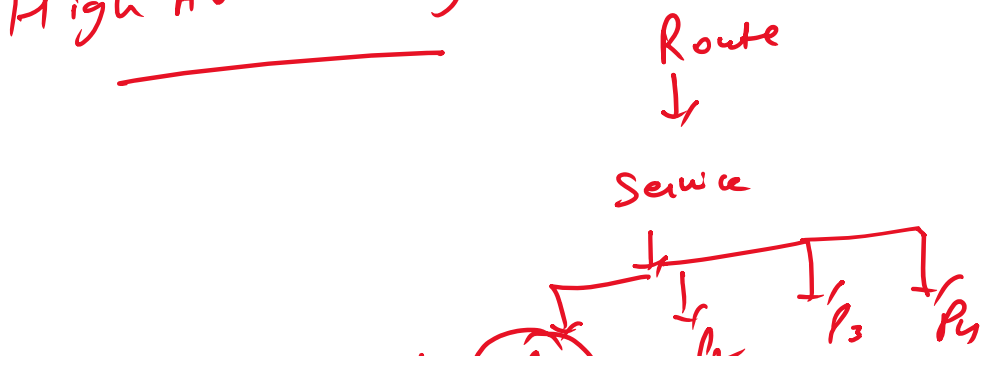
↓

Deployment: resource demo




```
cat <<'EOF' | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-cpu-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hpa-cpu-demo
  template:
    metadata:
      labels:
        app: hpa-cpu-demo
    spec:
      containers:
      - name: app
        image: registry.access.redhat.com/ubi9/ubi-minimal:latest
        command: ["/bin/sh", "-c"]
        args:
        - |
          echo "Starting CPU workload";
          while true;
          do
            :
          done
        resources:
          requests:
            cpu: "50m"
            memory: "64Mi"
          limits:
            cpu: "100m"
            memory: "128Mi"
EOF
```





```
cat <<'EOF' | oc apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
 name: ha-demo
spec:
 replicas: 3
 selector:
  matchLabels:
   app: ha-demo
 template:
  metadata:
   labels:
    app: ha-demo
  spec:
   containers:
   - name: app
    image: quay.io/openshift/origin-hello-openshift:latest
    ports:
    - containerPort: 8080
    resources:
     requests:
      cpu: "20m"
      memory: "32Mi"
     limits:
      cpu: "50m"
      memory: "64Mi"
EOF
```

Readiness Probe – Pod is ready to receive traffic?

Liveness Probe – Is Container is healthy or need restart?

Disruption – Pod will become Unavailable

- Involuntary → Unexpected fail
- Voluntary → Planned action

PDB → Pod Disruption Budget

Admin drain Node

↓

Openshift check PDB

↓

Will enough Pod Available?

Yes → Allow

No → Block / delay

```
cat <<'EOF' | oc apply -f -
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: ha-demo-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: ha-demo
EOF
```

→ PDB ↕ Pod

Min Available = 2

minAvailable : 2

maxUnavailable : 1

(5) Monitoring & Logging :-

1. Openshift Monitoring stack
2. Log via CLI
3. Metrics & Alert

(1) Why Monitoring & Logging Matter?

Monitor → Happening - Metrics, Health & Alert

Logging → Why it may be happening?

↳ Application runtime messages.

Alerting - Notification - Pod crash, CPU↑, Node Pressure,

Application Pod
↓ Logs
oc logs / Logging stack

Application Pod
↓ Metrics
Openshift Monitoring stack
↓
Prometheus / Dashboard / Alerts

① openshift Monitoring stack :-

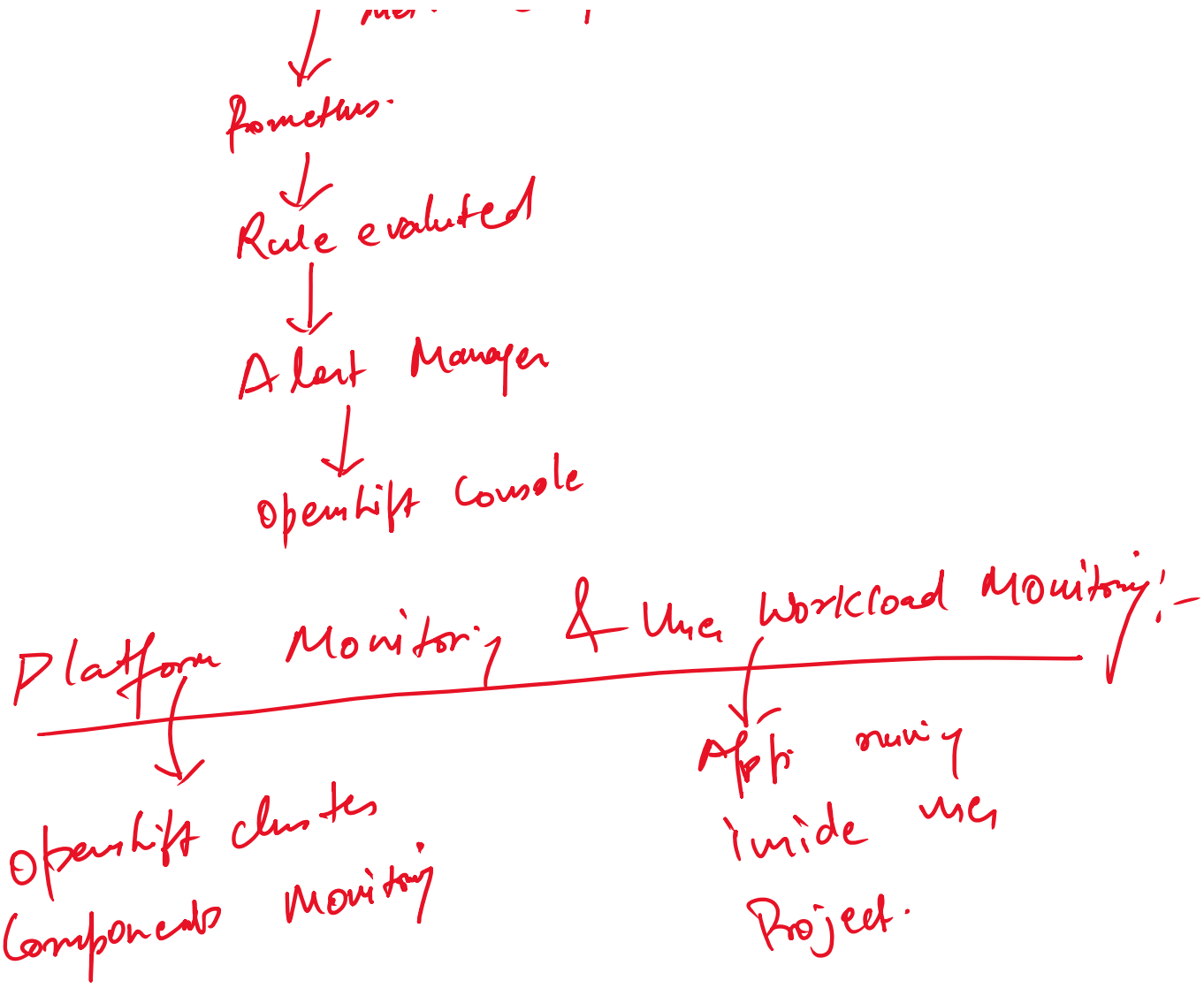
Built in Prometheus Based Monitoring System.

Cluster Nodes
Control Panel
Operator

Openshift Comp.
App workload.

① cluster Monitoring operator - Manage the Monitoring Stack
② Prometheus - collect & store the Metrics
③ Alert Manager - Handle Alert & Alert Routing
④ Thanos Querier - Query Access across Prometheus data.
⑤ Metric Target - Source where Metrics are scraped
⑥ Web Console overview - UI, Dashboard, Report & Alert.

Openshift components / App.
↓
Metrics endpoint




```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: log-demo-pod
  labels:
    app: log-demo
spec:
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      counter=1;
      while true;
      do
        echo "INFO: Application is running. Count=$counter";
        echo "DEBUG: Current time is $(date)";
        if [ $((counter % 5)) -eq 0 ]; then
          echo "WARN: This is a sample warning message";
        fi
        counter=$((counter+1));
        sleep 10;
      done
  restartPolicy: Never
EOF
```

Access Control deep dive:-

1. RBAC Advanced concept
2. Service Account usage.

1. RBAC → Role Based Access Control

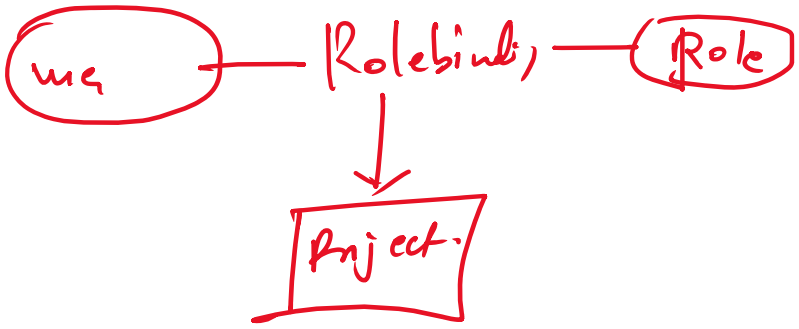
1. Role
2. clusterRole
3. RoleBinding.
4. clusterRoleBinding.

User / group / SA
↓
RoleBinding.
↓
Role
↓
Permission on resources

Vivek
↓
RoleBinding
↓
edit
↓
Can create Pod, Config, Secrets

1. **Role** → Gives permission inside the single Namespace.
2. **clusterRole** → Permission at the cluster level.

(3) Role Binding :-



(4) cluster Role Binding :- cluster scope.

(1) API group —
(2) resources - Pods
(3) verbs. - get, list, watch

RBAC verbs

get - View one object
list - View Multiple "
watch - Watch changes
create - Create resource
update - Modify resources (→ full)
patch - Modify Partical resource
delete - Delete resource (one)
deletecollection - Delete Multiple resources.

Common Built-in Roles in openshift

(1) View → Read-only Access
(2) edit → Create/update Most App. resources
(3) Admin → Manage Project Resources
(4) cluster-Admin → full cluster Access

```
cat <<'EOF' | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
EOF
```

Service Account :- Identify app. pods, operations & Automation.

User Account = human

Service Account = app / pod identify

Pod
↓
uses SA
↓
SA has Role binding
↓
Rolepoint point to Role
↓
Pod get limited Permission.

Pods use app-reader SA
↓
app bound pod-reader role
↓
Pod list pods
↓
Pod can't delete Pods.

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: sa-demo-pod
  labels:
    app: sa-demo
spec:
  serviceAccountName: app-reader
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      echo "ServiceAccount demo pod started";
      echo "Current Linux identity:";
      id;
      echo "ServiceAccount files:";
      ls -l /var/run/secrets/kubernetes.io/serviceaccount || true;
      while true;
      do
        sleep 30;
      done
  restartPolicy: Never
EOF
```

```
cat <<'EOF' | oc apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: no-token-pod
spec:
  automountServiceAccountToken: false
  containers:
  - name: app
    image: registry.access.redhat.com/ubi9/ubi-minimal:latest
    command: ["/bin/sh", "-c"]
    args:
    - |
      echo "Checking service account token path";
      ls -l /var/run/secrets/kubernetes.io/serviceaccount || true;
      sleep 3600;
  restartPolicy: Never
EOF
```

Tomorrow Topics:-

1. CI/CD
2. gitOps
3. optimization & cost awareness
4. cloud native openshift